OWNERSHIP — module ownership and PR rules

Locked: 2026-05-16 **Revisit at:** v1 retro (Jul 26), v2 retro (Sep 6)

Why this exists

Per Tadashi CV: "clear ownership" is a non-negotiable signal for elite hiring. When a senior engineer drills you on "what did *you* build?" the answer must be specific, defensible, and not a co-author claim. This file is the ground truth for that.

The split

Module	Owner	Co-owner / reviewer
Agent core (planner, router, state machine, tool-call contract, retry/recovery)	Marv	Ronin
Inference service (FastAPI/async, batching, MLX/llama.cpp serving, latency optimization)	Marv	Ronin
Telemetry (structured logging, OpenTelemetry traces, cost tracker)	Marv	Ronin
Eval harness (scenario format, runner, metrics, judge protocol, baseline vs candidate runner)	Ronin	Marv
Model training pipeline (data prep, SFT/LoRA, eval-during-training, model registry, ablations)	Ronin	Marv
Tool adapters (Notion, Google Calendar, Gmail, clipboard)	Ronin	Marv
Shared: repo scaffolding · `ARCHITECTURE.md` · final benchmark writeup · demo video	both	—

Person 1 = Marv Person 2 = Ronin

What "owner" means

For your owned modules:

- You make the architectural decisions, document them in `docs/decisions/NNNN-title.md` (ADR format), and defend them in review

- You write the core code yourself (CV: AI fine for boilerplate, glue, docs digestion; not for architecture/algorithms/data models)
- You write the tests
- You write the section in `ARCHITECTURE.md`
- Your CV bullet for this module says "designed and built" — the co-owner's CV bullet for the same module says "contributed to" or omits it

For your co-owned modules:

- You review every PR; the owner cannot merge without your approval
- You can propose changes via PR but cannot merge without the owner's approval
- You ask questions; the owner answers and documents

PR rules

1. `main` is protected. No direct pushes. No force-push to `main`, ever.
2. Every PR closes one tracked GitHub Issue, scoped < 2 days of work.
3. Every PR requires the relevant co-owner's review and approval before merge.
4. CI (ruff + pyright + pytest) must be green to merge.
5. PR description includes: what changed, why, how to test, and explicit "I wrote this" or "AI-assisted with: [scope]" disclosure.
6. Per CV rule: if AI wrote code, the human must be able to explain it line-by-line on request. Don't merge code you can't defend.
7. No `--no-verify`. No skipping hooks.
8. Squash merge to keep `main` history clean.

Conflict resolution

When the two of you disagree on an architecture choice that crosses owned modules:

1. Each writes a short doc (≤300 words) stating their proposal + tradeoff
2. Pick the one with the clearer measurable consequence (e.g. "this design lets us swap inference backends in <100 LOC vs ~1000 LOC")
3. If still tied, the owner of the module most affected gets the call
4. Document the decision in an ADR

Self-honesty audit

Once a month, each person writes a 200-word retro on what they actually built vs what they shipped via AI. The CV explicitly warns: "if you cannot rewrite the critical part yourself if needed, then AI is currently weakening you." Catch this early.

When this file changes

Any scope change to an owned module requires a PR that updates this file. No silent ownership drift.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.